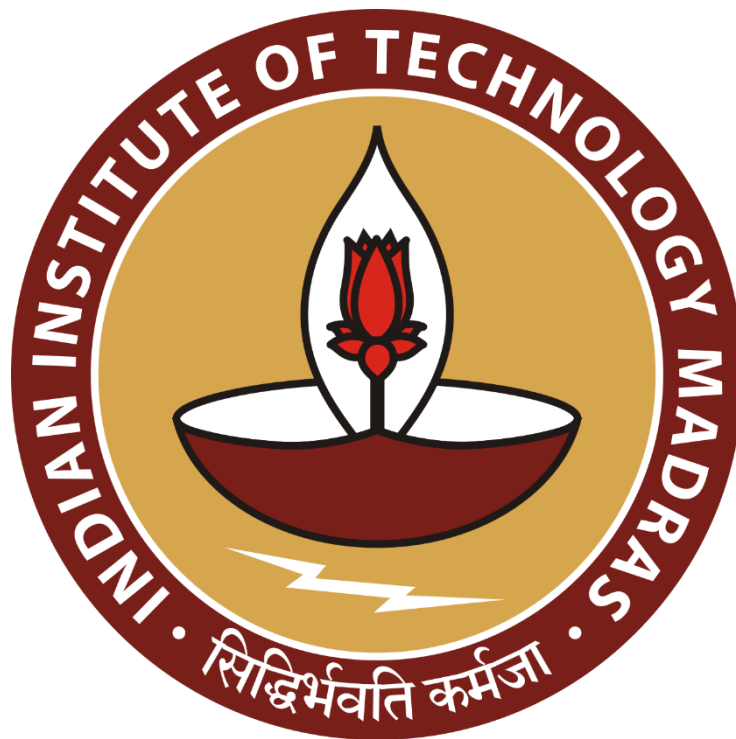




Milestone 4 Report: Formulating MCQ Task & Fine-Tuning



JULY 16, 2026
NAME: JOB BENNY NAMBERIL
Roll Number: 22f3000174

Milestone Report: Formulating MCQ Task & Fine-Tuning

Objective

The objective of this milestone was to formulate the Smart MCQ Solver Challenge as a multiple-choice classification problem using transformer-based language models. The work involved converting the dataset into the format expected by Hugging Face multiple-choice models, applying Low-Rank Adaptation (LoRA) for efficient fine-tuning, configuring a lightweight training pipeline using the Hugging Face Trainer API, and understanding techniques for managing GPU memory and improving training efficiency.

Data Formatting for Multiple-Choice Classification

The original dataset contains a question prompt, five answer options (A, B, C, D, and E), and the correct answer. Unlike standard text classification tasks, multiple-choice transformer models require each answer option to be paired individually with the corresponding question.

For every question, five separate input sequences were created using the following format:

Prompt [SEP] Option A

Prompt [SEP] Option B

Prompt [SEP] Option C

Prompt [SEP] Option D

Prompt [SEP] Option E

The [SEP] token acts as a separator between the question and each answer option, allowing the transformer model to understand both pieces of information simultaneously. Each answer label was encoded into numeric form using the mapping A→0, B→1, C→2, D→3, and E→4 so that the model could compute supervised training loss.

Tokenization for Multiple-Choice Models

The formatted question-option pairs were tokenized using the bert-base-uncased tokenizer. During tokenization, padding was applied to a fixed maximum sequence length while longer sequences were truncated to ensure uniform tensor dimensions.

For a single question, the tokenized inputs were stored as tensors of shape [5, sequence_length], representing the five answer choices. During batch processing, the model receives tensors with dimensions [batch_size, 5, sequence_length]. This structure enables the model to evaluate all answer options simultaneously for every question.

Multiple-Choice Classification Model

The Hugging Face AutoModelForMultipleChoice architecture was used for this milestone. Unlike standard classification models that output a single prediction, this architecture produces one logit score for each answer choice. Therefore, every question generates five logits corresponding to options A through E.

These logits are passed through the Softmax function to obtain probability distributions across all answer choices. The option with the highest probability is selected as the model's prediction. During supervised learning, the encoded numeric label is used to compute cross-entropy loss, allowing the model to update its parameters based on prediction errors.

Introduction to LoRA Fine-Tuning

To improve training efficiency, Low-Rank Adaptation (LoRA) was applied to the pretrained BERT model. Instead of updating every parameter in the network, LoRA freezes the original pretrained weights and inserts small trainable adapter matrices into selected attention layers.

The LoRA configuration used during this milestone consisted of:

- Rank (r): 8
- LoRA Alpha: 16
- Target Modules: Query and Value projection layers
- LoRA Dropout: 0.1
- Bias: None
- Task Type: Sequence Classification

With this configuration, only the adapter parameters are updated during training while the pretrained BERT parameters remain unchanged.

Advantages of LoRA over Full Fine-Tuning

LoRA provides several advantages compared to traditional full fine-tuning. Since only a small number of parameters are trained, GPU memory consumption is significantly reduced. Training becomes faster because gradients are computed only for the adapter layers rather than the entire model. The resulting checkpoints are much smaller, making them easier to store and share. LoRA also enables efficient experimentation on consumer-grade GPUs where full fine-tuning may not be practical. Overall, LoRA achieves competitive performance while requiring substantially fewer computational resources.

Setting Up the Training Pipeline

The Hugging Face Trainer API was used to simplify the fine-tuning process. The overall training pipeline consisted of loading the dataset, encoding the answer labels, formatting prompt-option pairs, tokenizing the data, applying LoRA adapters, configuring training arguments, and finally performing supervised fine-tuning.

For demonstration purposes, only the first 32 training examples were used with the following configuration:

- Maximum sequence length: 64
- Batch size per device: 4
- Gradient accumulation steps: 1
- Maximum training steps: 4

The Trainer API automatically handled batching, optimization, loss computation, gradient updates, checkpoint management, and logging throughout the training process.

Managing GPU Memory

Transformer models are computationally intensive and require careful memory management. Several strategies were implemented during this milestone to ensure successful training within available GPU resources.

The batch size was reduced to four samples per iteration to minimize memory consumption. The maximum sequence length was lowered from 128 to 64 tokens during the tiny fine-tuning experiment, further reducing memory requirements. LoRA itself significantly decreased GPU memory usage because only the adapter parameters required gradient storage and optimization. Gradient accumulation was configured to one for this experiment but can be increased in larger training scenarios to simulate larger effective batch sizes without exceeding memory limitations.

Training Efficiency Strategies

Several Hugging Face training arguments were selected to improve training efficiency. Setting `max_steps = 4` enabled quick experimentation without completing a full epoch over the dataset. A batch size of four balanced computational efficiency with available GPU memory. Logging was configured to occur after every optimization step, allowing continuous monitoring of training progress. The parameter `remove_unused_columns = False` ensured that all multiple-choice input features remained available during training, while external logging services were disabled by setting `report_to = "none"`.

These configurations provided a lightweight yet effective training environment suitable for experimentation and educational purposes.

Results

The milestone successfully demonstrated the complete workflow for multiple-choice transformer models. The dataset was transformed into the required multiple-choice format, answer labels were numerically encoded, and tokenized inputs were generated with the correct tensor dimensions. The `AutoModelForMultipleChoice` architecture produced logits for each answer option and computed supervised loss using the encoded labels. LoRA adapters were successfully integrated into the pretrained BERT model, enabling parameter-efficient fine-tuning. A small training experiment using the Hugging Face Trainer verified that the complete pipeline functioned correctly, and inference was performed by applying Softmax to the model logits to obtain probability distributions over the five answer choices.

Challenges Encountered

Several practical challenges were encountered during implementation. Properly formatting the prompt-option pairs required careful preprocessing to match the expected input structure of the multiple-choice model. Ensuring that tensor dimensions remained consistent throughout tokenization and batching was essential for successful model execution. Version compatibility issues involving PyTorch, Torchvision, and the Hugging Face Datasets library required troubleshooting to resolve import errors. Additionally, GPU memory limitations highlighted the

importance of selecting appropriate batch sizes, sequence lengths, and efficient fine-tuning techniques such as LoRA.

Conclusion

This milestone established the complete workflow for solving multiple-choice question answering tasks using transformer-based language models. The dataset was successfully reformatted into the structure required by Hugging Face multiple-choice architectures, enabling efficient processing of question-answer pairs. The application of LoRA demonstrated that parameter-efficient fine-tuning can significantly reduce computational cost while maintaining effective model adaptation. Furthermore, GPU memory optimization techniques and carefully selected training arguments ensured that the fine-tuning process remained practical even on limited hardware resources. Overall, this milestone provided a strong foundation for developing scalable, efficient, and accurate transformer-based multiple-choice question answering systems that can be extended to larger datasets and more advanced pretrained language models in future work.